CS 6650 – Assignment 6: Performance Bottlenecks & Horizontal Scaling

Eroniction Presley

February 23, 2026

1 Part II: Identifying Performance Bottlenecks

1.1 Service Implementation

The product search service was built in Go using the standard `net/http` library. At startup, 100,000 products are generated and stored in a `sync.Map` for thread-safe concurrent access. Each product contains an ID, Name, Category, Description, and Brand. Names follow the format `"Product [Brand] [ID]"` and categories rotate through 10 options (Electronics, Books, Home, Sports, etc.) using modulo arithmetic to ensure consistent search behavior.

The search endpoint at `/products/search?q={query}` performs case-insensitive matching against both the product name and category fields. Each search checks exactly 100 products and then stops, simulating a fixed-cost computation (similar to running an AI model or processing video frames). A counter is incremented for every product checked, not just matches. The endpoint returns a maximum of 20 matching results along with the total match count and search duration.

A `/health` endpoint returns a simple 200 OK response for load balancer health checks.

1.2 Deployment

The service was containerized using a multi-stage Docker build with `golang:1.24-alpine` as the build stage and `alpine:latest` as the runtime. The Docker image was pushed to Amazon ECR and deployed on ECS Fargate with the following configuration:

- **CPU**: 256 units (0.25 vCPU)
- **Memory**: 512 MB
- **Instances**: 1
- **Network**: Public subnet with auto-assigned public IP

All infrastructure was provisioned using Terraform, including the ECS cluster, task definition, service, security groups, and CloudWatch log group. The LabRole IAM role was used for task execution permissions.

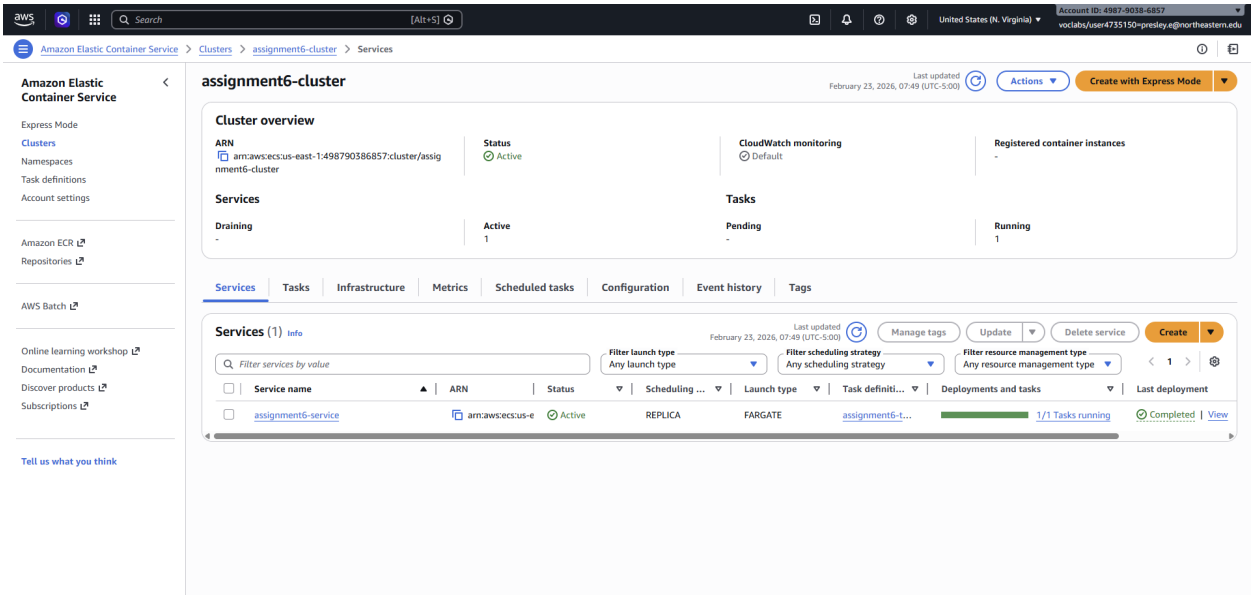

Figure 1: ECS service running with 1 task in Part II

1.3 Verification

Before load testing, the service was verified with single requests using `curl`:

```
curl http://<PUBLIC_IP>:8080/products/search?q=alpha
curl http://<PUBLIC_IP>:8080/health
```

The search endpoint returned JSON with matching products, confirming correct behavior. Response time for a single request was under 20ms, and the health check returned 200 OK.

```
PS D:\BSDS\Assignment 6> curl http://98.82.163.196:8080/products/search?q=alpha

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
     RECOMMENDED ACTION:
     Use the -UseBasicParsing switch to avoid script code execution.

     Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): yes


StatusCode        : 200
StatusDescription : OK
Content           : {"products":[{"id":39760,"name":"Product Alpha 39760","category":"Electronics","description":"A great Electronics product by Alpha","brand":"Alpha"},{"id":38880,"name":"Product
                    Alpha 38880","category"...
RawContent        : HTTP/1.1 200 OK
                    Transfer-Encoding: chunked
                    Content-Type: application/json
                    Date: Mon, 23 Feb 2026 12:33:02 GMT

                    {"products":[{"id":39760,"name":"Product Alpha 39760","category":"Electronics","desc...
Forms             : {}
Headers           : {[Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Mon, 23 Feb 2026 12:33:02 GMT]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2503
```

Figure 2: Search endpoint verification via curl

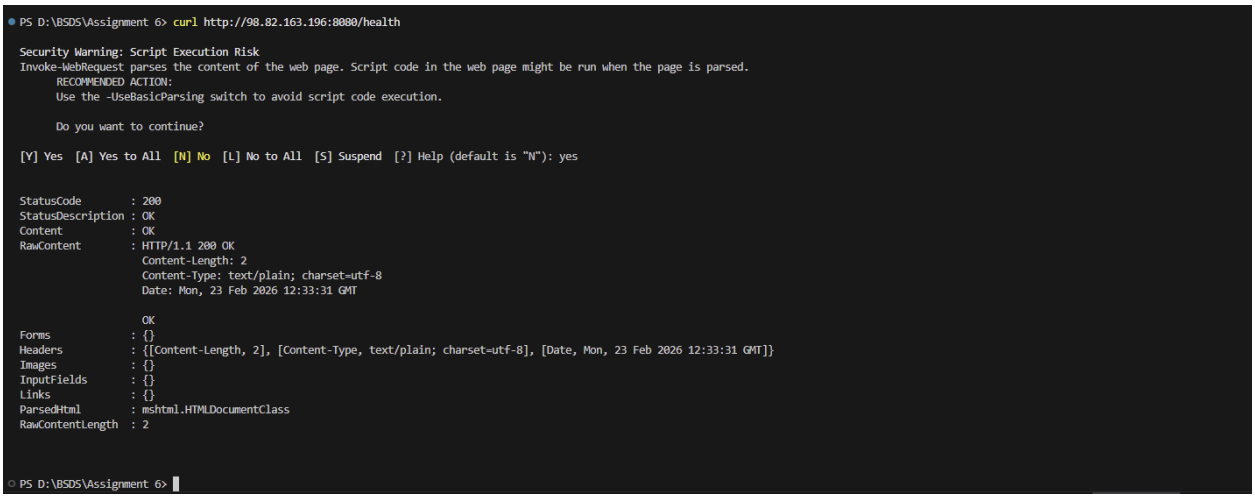

Figure 3: Health endpoint verification via curl

1.4 Load Testing Results

Load tests were conducted using Locust with `FastHttpUser` and no wait time between requests, maximizing the load on the service. Search terms were randomly selected from `["alpha", "beta", "electronics", "books", "home"]`.

1.4.1 Test 1 – Baseline (5 users, 2 minutes)

The system handled the baseline load comfortably:

- **Throughput**: 180 requests/second
- **p50 response time**: 27ms
- **p95 response time**: 38ms
- **Failures**: 0 (0.00%)

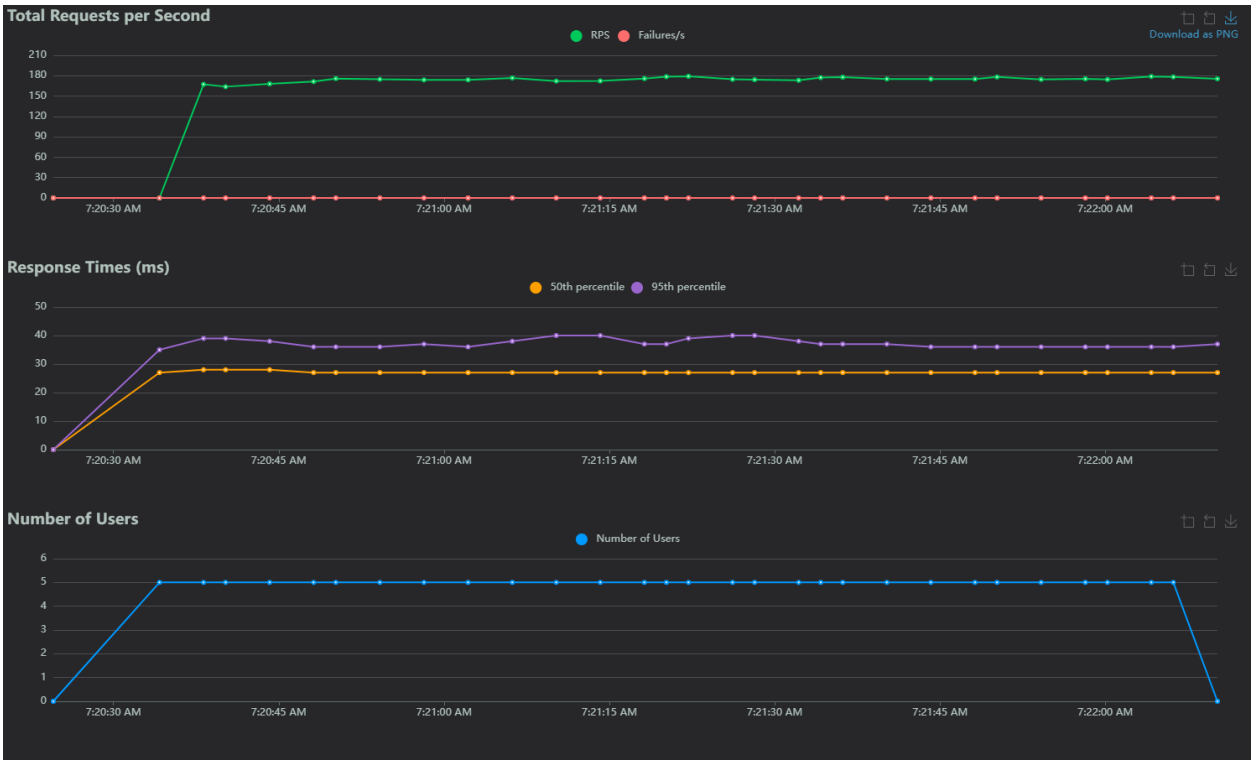

Figure 4: Baseline: 5 users for 2 minutes

1.4.2 Test 2 – Increased Load (20 users, 3 minutes)

Throughput scaled up but response times began to degrade:

- **Throughput**: 560 requests/second
- **p50 response time**: 35ms
- **p95 response time**: 95ms at peak
- **Failures**: 0 (0.00%)

The 3x increase in throughput came with a 2.5x increase in p95 latency, indicating the single instance was approaching its capacity.

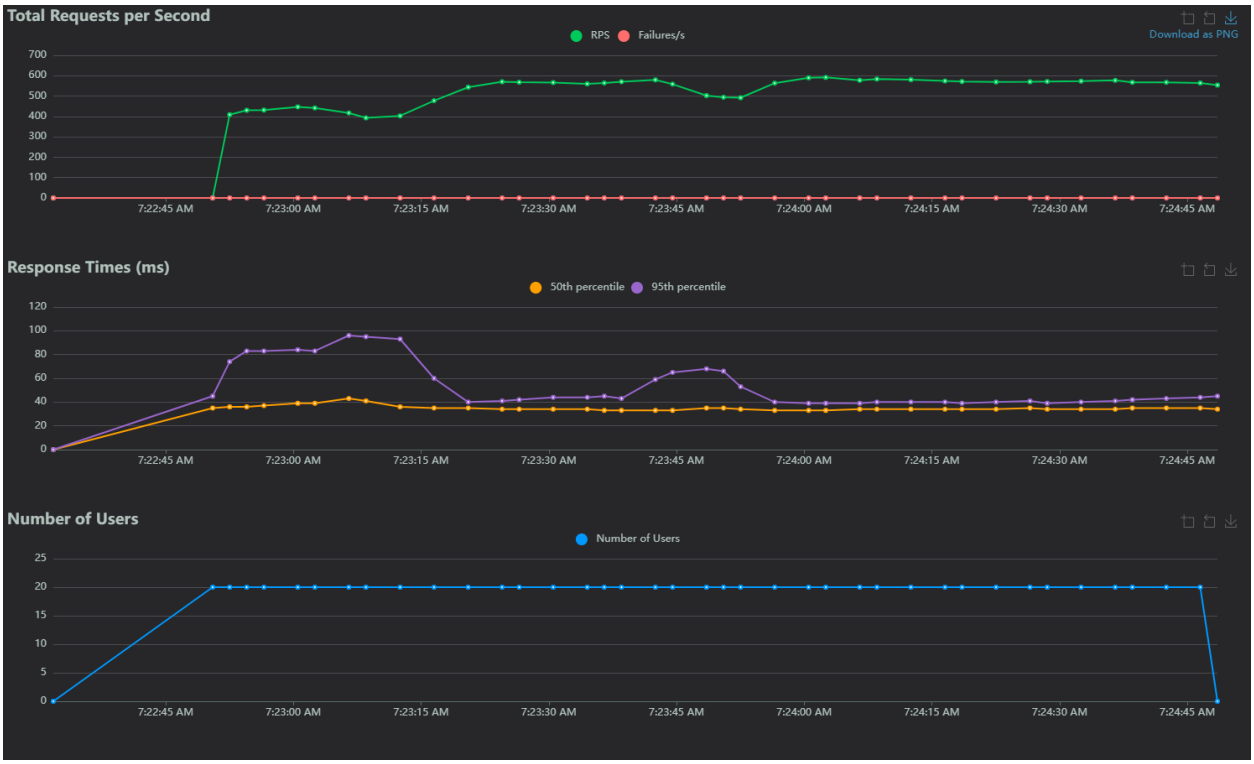

Figure 5: Increased load: 20 users for 3 minutes

1.5 CloudWatch Metrics

CloudWatch metrics confirmed that CPU utilization increased proportionally with load while memory utilization remained flat. This is expected because all 100,000 products are loaded into memory at startup and no additional allocations occur during search operations. The search logic is purely CPU-bound – each request performs string comparisons across 100 products.

1.6 Analysis: Optimization vs. Scaling

Which resource hits the limit first? CPU is the clear bottleneck. Memory remained constant throughout testing since the product data is loaded once at startup and the search operation does not allocate significant additional memory.

How much did response times degrade? The p95 response time increased from 38ms (5 users) to 95ms (20 users), a 2.5x degradation. The p50 also rose from 27ms to 35ms, showing that even median requests were affected.

Could doubling CPU (256 → 512 units) help? Yes. Since CPU is the bottleneck and the search logic performs a fixed amount of work per request (checking exactly 100 products), doubling CPU would roughly double the service's capacity. This is vertical scaling – giving a single instance more resources.

The key insight: The search algorithm is already bounded (100 products per search). There is no code optimization that would meaningfully reduce per-request CPU usage. The evidence for this is that memory is stable and CPU scales linearly with load. When you see this pattern – CPU-bound with no optimization headroom – the answer is **more compute power**, not better code. CloudWatch metrics showing rising CPU alongside stable memory are the signal to scale,

not optimize.

2 Part III: Horizontal Scaling with Auto Scaling

2.1 Infrastructure Changes

To solve the bottleneck from Part II, the same Go service (unchanged) was redeployed with horizontal scaling infrastructure:

- **Application Load Balancer (ALB)**: Internet-facing, listens on port 80 and forwards to the target group
- **Target Group**: IP-based (required for Fargate), port 8080, health checks on `/health` every 30 seconds with a healthy threshold of 2 consecutive successes
- **Auto Scaling Policy**: Target tracking on `ECSServiceAverageCPUUtilization` at 70%, min 2 instances, max 4 instances, 300-second cooldown for both scale-out and scale-in
- **Security Groups**: ALB security group allows inbound on port 80; ECS security group only allows inbound on port 8080 from the ALB security group, ensuring containers are not directly accessible

The entire infrastructure was provisioned via Terraform in a single `main.tf` file. The ECS service was configured with `desired_count = 2` to start with redundancy from the beginning.

2.2 Load Testing Results

All Part III tests hit the ALB DNS endpoint rather than individual task IPs. The system was tested at progressively higher loads to observe scaling behavior:

Users	RPS	p50 (ms)	p95 (ms)	Failures
20	560	35	45	0%
50	1,300	35	45	0%
200	3,200	50	120	0%
500	4,000	100	300	0%

Table 1: Load test progression with horizontal scaling

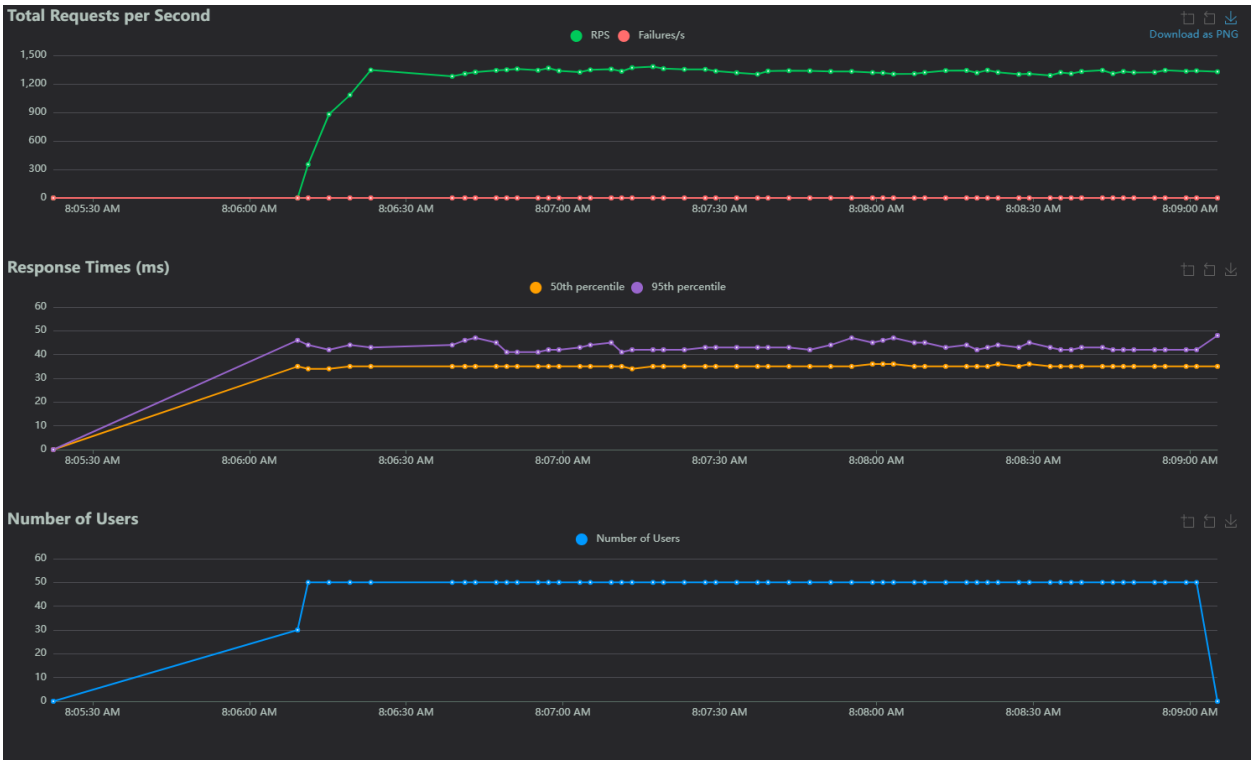

Figure 6: 50 users with horizontal scaling

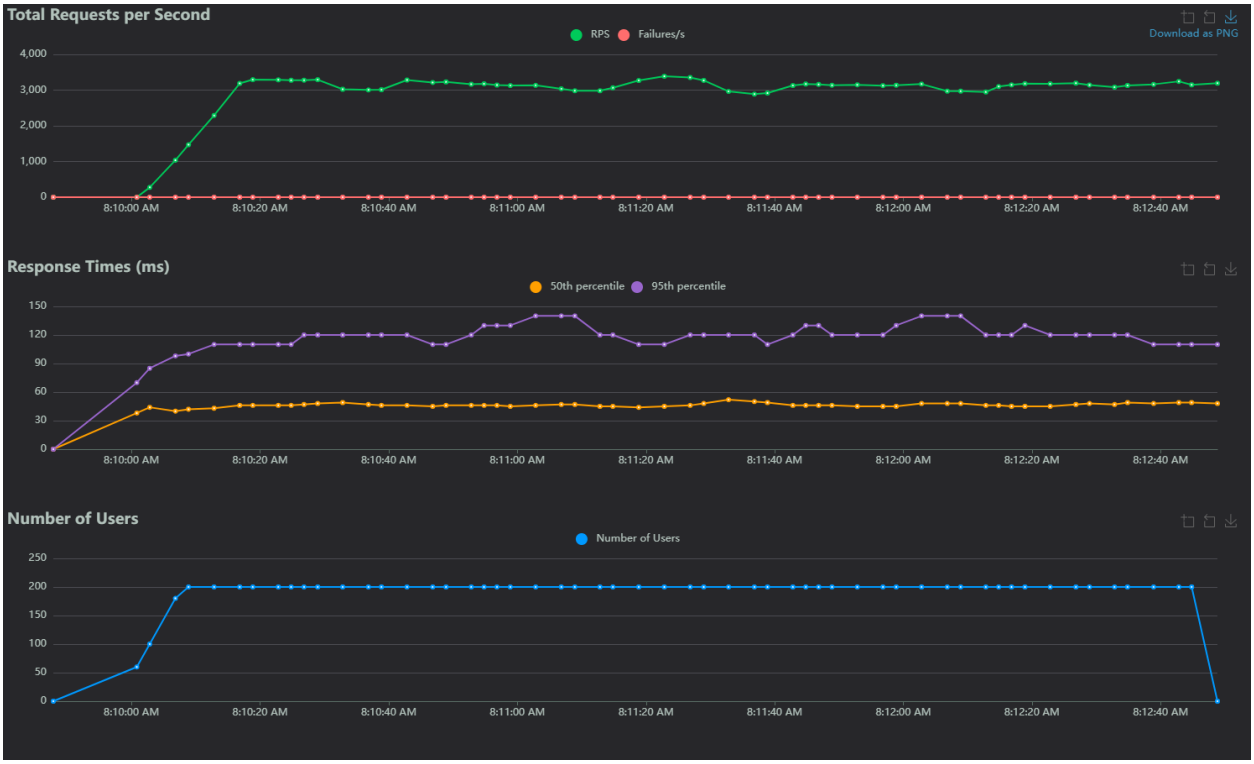

Figure 7: 200 users with horizontal scaling

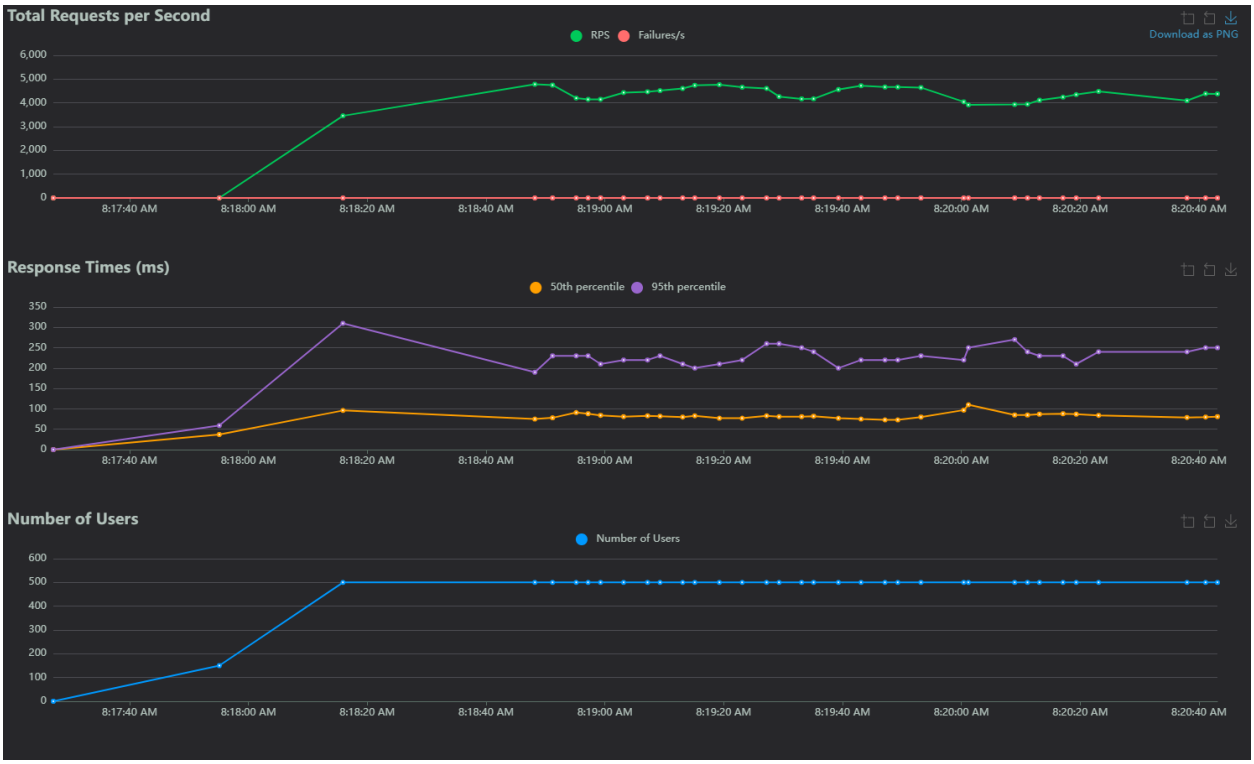

Figure 8: 500 users with horizontal scaling

At 500 concurrent users, the system sustained 4,000 RPS with zero failures. The 500-user test generated a Locust warning about local CPU usage exceeding 90%, meaning the bottleneck at that point was actually the load-testing machine, not the server infrastructure.

2.3 Auto Scaling in Action

During the 500-user test, CloudWatch detected average CPU utilization exceeding the 70% target. Auto scaling responded by increasing the desired task count from 2 to 3 instances.

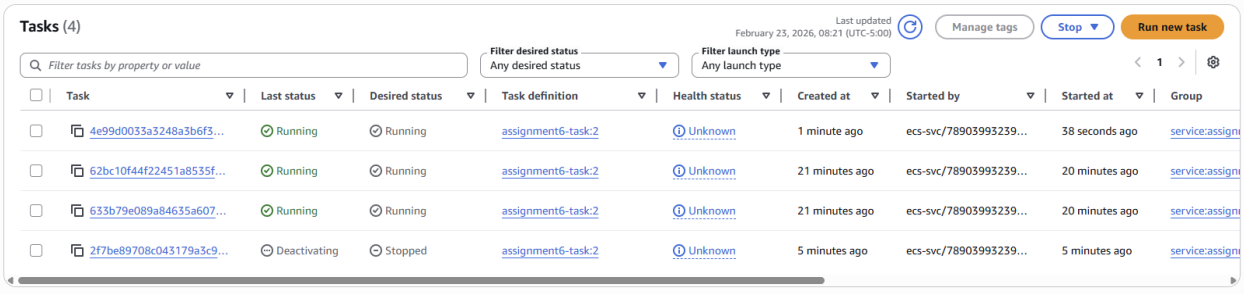

Figure 9: ECS auto scaling: 3 running tasks after CPU threshold exceeded

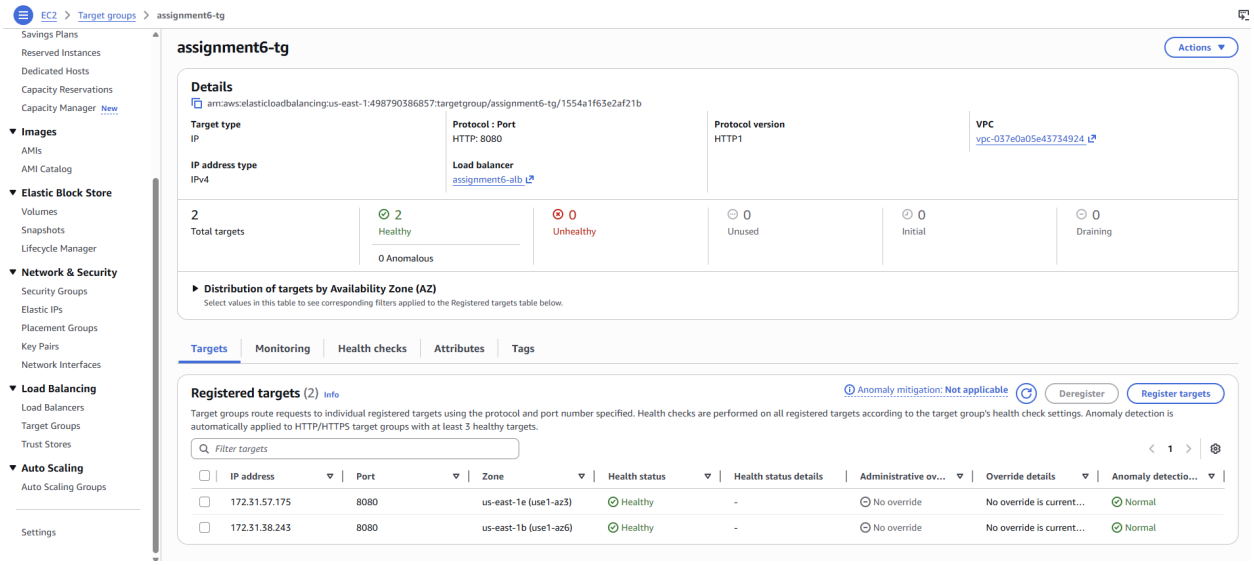

Figure 10: Target group showing healthy targets

The new instance took approximately 1 minute to launch, pull the Docker image, start the container, generate 100,000 products, and pass 2 consecutive health checks. Once healthy, the ALB immediately began routing traffic to it.

2.4 Resilience Testing

During a 500-user load test, one running ECS task was manually stopped via the ECS Console to simulate an instance failure. The following was observed:

- The ALB detected the unhealthy target and immediately stopped routing traffic to it
- The remaining healthy instances absorbed the full load
- ECS recognized the desired count was no longer met and launched a replacement task within 1 minute
- Locust reported **zero failures** and **zero request errors** throughout the entire event
- Response times showed a brief spike as load was redistributed, then stabilized once the replacement task came online

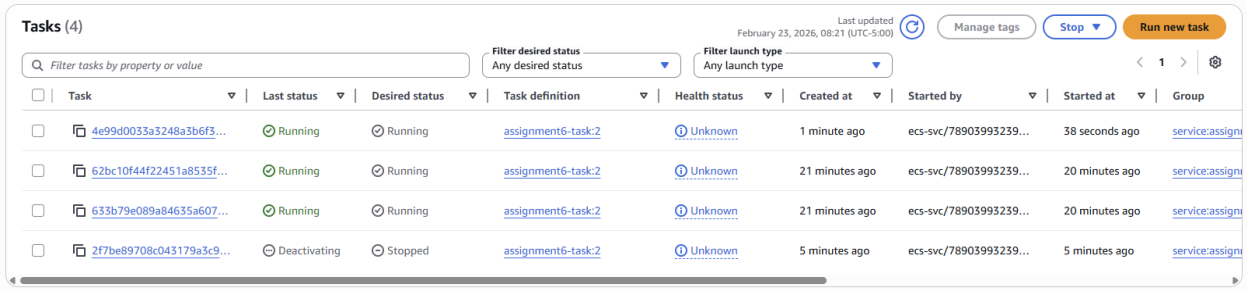

Figure 11: ECS Tasks: stopped task (Deactivating) and automatic replacement (1 minute ago)

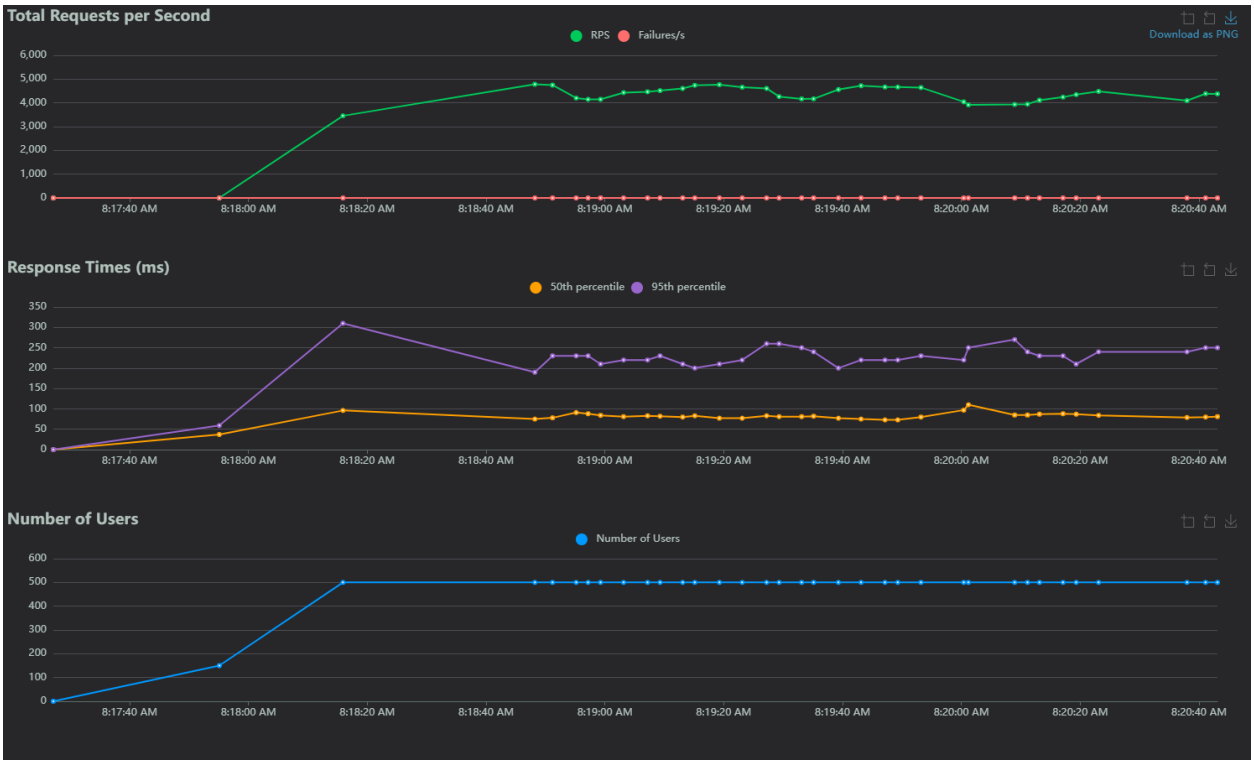

Figure 12: Locust graphs during resilience test – zero failures

This demonstrates a key advantage of horizontal scaling: individual instance failures do not bring down the service. The combination of ALB health checks, target group management, and ECS desired count maintenance provides self-healing behavior.

2.5 CloudWatch Metrics – Horizontal Scaling

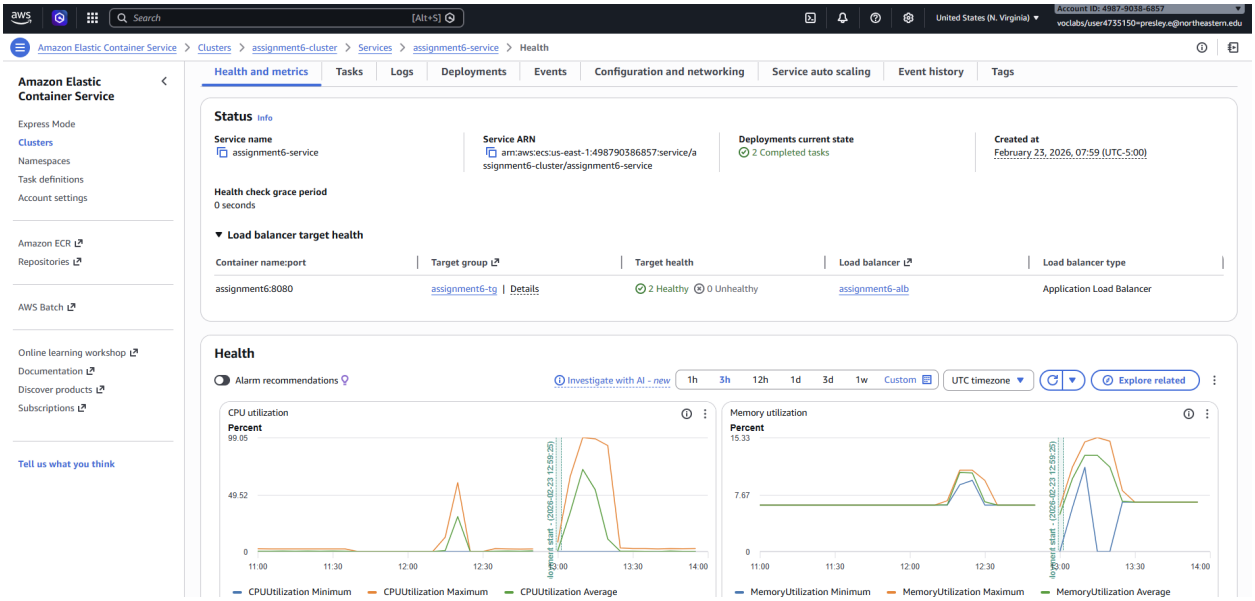

Figure 13: CloudWatch CPU utilization (99% peak) and Memory utilization (7.67% steady) during Part III

CPU utilization spiked to 99% during the 500-user tests, confirming CPU as the bottleneck. Memory remained steady at 7.67%, consistent with Part II findings.

2.6 Component Roles

Application Load Balancer (ALB) serves as the single entry point for all client traffic. It distributes incoming HTTP requests across all registered healthy targets using round-robin routing. Clients interact with a single DNS name and are completely unaware of how many backend instances exist or which instance handles their request.

Target Group maintains a registry of available backend instances and continuously monitors their health. Every 30 seconds, it sends a request to `/health` on each registered target. After 2 consecutive successful responses, a target is marked healthy and receives traffic. After 3 consecutive failures, the target is removed from rotation. This ensures requests never reach unhealthy instances.

Auto Scaling continuously monitors the average CPU utilization across all running tasks via CloudWatch. When CPU exceeds the 70% target, the scaling policy increases the desired task count (up to the max of 4). When load decreases and CPU falls below the target, tasks are removed (down to the min of 2). The 300-second cooldown prevents rapid oscillation between scaling events.

2.7 Vertical vs. Horizontal Scaling

Aspect	Vertical Scaling	Horizontal Scaling
Approach	Bigger instance	More instances
Complexity	Low (config change)	Higher (ALB, health checks)
Fault tolerance	Single point of failure	Survives instance failures
Cost efficiency	Pay for peak capacity	Scale with demand
Upper limit	Hardware maximum	Theoretically unlimited
Downtime	May require restart	Zero-downtime scaling

Table 2: Vertical vs. Horizontal scaling comparison

For this workload, horizontal scaling is the superior approach because: (1) the service is stateless – each instance independently loads products at startup with no shared state; (2) the workload is CPU-bound and easily parallelizable across instances; (3) fault tolerance is critical for production services; (4) auto scaling allows cost efficiency by matching capacity to demand.

2.8 Scaling Predictions

Based on the observed data, each 0.25 vCPU instance handles approximately 1,500–2,000 RPS at acceptable latency (<50ms p50).

Gradual load increase: Auto scaling responds smoothly. As CPU crosses 70%, new instances are added and the ALB distributes load evenly. Response times remain stable because capacity grows with demand.

Sudden traffic spike (e.g., 0 to 500 users instantly): There will be a brief latency spike lasting 1 minute while new tasks start, generate products, and pass health checks. The 300-second cooldown prevents premature scale-in if the spike is sustained.

Sustained high load beyond max instances: With 4 instances at max capacity, each handling 2,000 RPS, the system can sustain 8,000 RPS. Beyond this, the max instance count would need to be increased. CloudWatch alarms could be configured to alert when the system is running at maximum capacity.

Load decrease: After load drops, the 300-second scale-in cooldown ensures instances are not removed prematurely. Once CPU consistently falls below 70%, tasks are gradually removed back to the minimum of 2, maintaining baseline redundancy.

3 Conclusion

This assignment demonstrated the complete lifecycle of identifying and solving performance bottlenecks in a distributed system. In Part II, load testing and CloudWatch metrics revealed that CPU was the bottleneck for a fixed-cost computation workload, and no code optimization could reduce per-request cost. In Part III, horizontal scaling with an ALB, target groups, and auto scaling solved the bottleneck while also providing fault tolerance through automatic instance replacement. The resilience test confirmed that the system self-heals from instance failures with zero client-visible errors. These patterns – metric-driven scaling decisions, horizontal scaling, and self-healing infrastructure – are foundational to modern distributed systems architecture.